Clustering dans les RCSF : simulation de LEACH sous NS3

Rapport de TP – Réseaux sans fil

KOURA Lémiyi Stéphane Ulrich (SIE) · M1 IBAM / UJKZ · Octobre 2026

Code, résultats et figures : `https://github.com/lemiyi/tp-reseau-sans-fil`

1 Introduction

Dans un réseau de capteurs sans fil (RCSF), la batterie n'est pas remplaçable : la durée de vie dépend donc de la répartition des transmissions, dont le coût croît en d^2 puis en d^4 avec la distance. Le clustering réduit ce coût : les membres d'un cluster envoient leurs données à un chef de cluster (CH) qui les agrège et les transmet à la station de base (BS). **LEACH** (Heinzelman *et al.* [1,2]) élit les CH de façon probabiliste : au round r, un nœud qui n'a pas encore été CH dans l'époque courante ($1/p$ rounds) le devient s'il tire un nombre inférieur au seuil

$$T(n) = \frac{p}{1 - p\,(r \bmod 1/p)}.$$

Un round comprend une phase d'initialisation (annonce ADV des CH, adhésion JOIN des membres au CH le plus proche, planning TDMA SCHEDULE) puis un régime permanent (membres → CH, agrégation, CH → BS).

L'objectif du TP est de vérifier par simulation les affirmations du tableau comparatif du cours (consommation d'énergie, durée de vie, passage à l'échelle) et de montrer que LEACH est « un piège » : son élection ignore l'énergie résiduelle et la distance à la BS.

Choix de modélisation. Nous utilisons NS3 3.40 (modules *core*, *network* et *mobility*), installé dans une VM Ubuntu 22.04 ARM64 (Vagrant + QEMU) sur Mac Apple Silicon. NS3 fournit les nœuds, leurs positions, le temps simulé et le générateur aléatoire ; l'énergie suit exactement le modèle de premier ordre imposé, codé dans `leach.cc` (444 lignes : `ElectCH`, `FormClusters`, `SteadyState`, `Record`). Le canal est idéal, avec une option de pertes aléatoires. Cet « niveau applicatif » donne la maîtrise totale des valeurs imposées et une simulation rapide, mais ne modélise ni collisions ni interférences (voir §4).

2 Configuration et validation

Paramètre	Valeur	Paramètre	Valeur
Nœuds N	100 (50 et 300 pour l'échelle)	Zone	200×200 m, positions aléatoires fixes
Station de base	$(100\,;\,250)$, hors de la zone	Énergie initiale	0,5 J par nœud (50 J au total)
Rounds	1000 (1 round = 1 s simulée)	Graines	5 (`RngRun` 1 à 5, `SetSeed` 12345)
E_{elec} ; ε_{fs}	50 nJ/bit ; 10 pJ/bit/m^2	ε_{mp} ; d_0	0,0013 **pJ**/bit/m^4 ; 87,7 m
E_{DA}	5 nJ/bit/signal	Paquets données / contrôle	4000 bits / 200 bits
p	0,05 (époque de 20 rounds)	Débit radio ; portée ADV	250 kbit/s ; 100 m
Repli si aucun CH	envoi direct à la BS	Pertes aléatoires	0 % (variante : 2 %)

Table 1 – Paramètres de la simulation (imposés : N, zone, BS, énergie, rounds, modèle radio ; les autres sont des choix).

Le modèle d'énergie est le modèle radio de premier ordre : pour émettre k bits sur une distance d,

$$E_{Tx}(k,d) = \begin{cases} E_{\text{elec}}\,k + \varepsilon_{\text{fs}}\,k\,d^2 & \text{si } d < d_0, \\ E_{\text{elec}}\,k + \varepsilon_{\text{mp}}\,k\,d^4 & \text{si } d \geq d_0, \end{cases} \qquad E_{Rx}(k) = E_{\text{elec}}\,k, \qquad d_0 = \sqrt{\varepsilon_{\text{fs}}/\varepsilon_{\text{mp}}}.$$

Unité de ε_{mp}. L'énoncé donne 0,0013 nJ/bit/m^4. Avec cette valeur, $d_0 = \sqrt{10\,\text{pJ}/1{,}3\,\text{nJ}} \approx 2{,}8$ m et presque toutes les liaisons seraient en d^4, avec un coût 1000 fois trop élevé. Nous retenons donc 0,0013 **pJ**/bit/m^4 (valeur de la littérature LEACH, $d_0 = 87{,}7$ m, cohérent avec une zone de 200 m) ; ce choix reste à confirmer par l'enseignant.

Validation.

(i) Calculs à la main (paquet de 4000 bits), recoupés par script : membre → CH à 30 m = 236 µJ ; CH → BS à 150 m = 2,83 mJ, à 200 m = 8,52 mJ, à 270 m = 27,8 mJ (118 fois le coût membre → CH) ; réception d'un paquet = 200 µJ.

(ii) Round 0 (1 CH, 99 membres) : 89,6 mJ consommés, dont ≈ 42 mJ indépendants de la distance (émission 19,8 + réception 19,8 + agrégation 2,0 mJ) ; le reste est l'amplification et le contrôle.

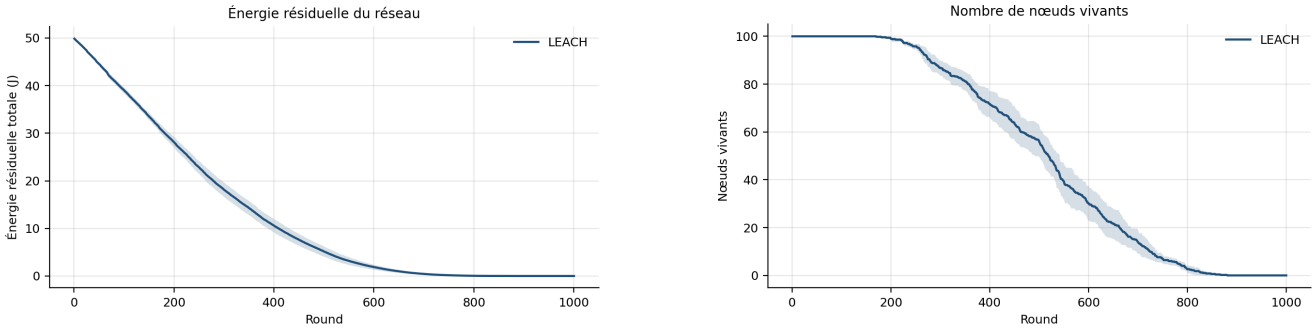
(iii) Une réimplémentation Python indépendante (`ref_leach.py`, 10 graines) donne FND ≈ 206 et HND ≈ 526, contre 191 et 519 sous NS3 (écarts relatifs de 7 % et 1 %), très en dessous du facteur 2 qui révélerait une erreur.

(iv) Contrôle des 20 CSV : 1000 lignes, aucune valeur manquante, énergie totale non croissante.

3 Résultats

Toutes les valeurs sont des moyennes sur **5 graines** $\pm$ écart-type (bandes ombrées des figures) ; 1 round = 1 s simulée.

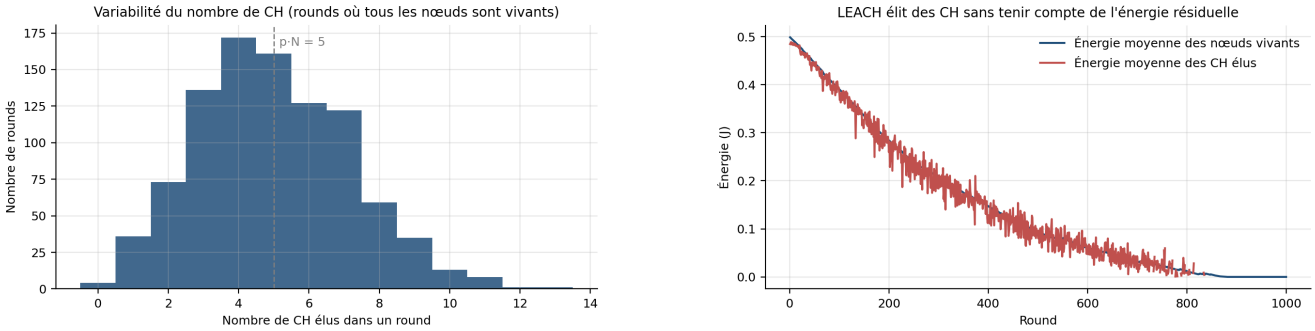
3.1 Expérience A : énergie et durée de vie ($N = 100$)



(a) Énergie résiduelle totale (J) **(b)** Nombre de nœuds vivants

FIGURE 1 – Expérience A : énergie du réseau et nœuds vivants en fonction du round ($N = 100$, 5 graines).

Les valeurs par graine figurent en annexe ; en moyenne FND $= 191 \pm 14$, HND $= 519 \pm 29$ et LND $= 844 \pm 31$ rounds. L'énergie baisse presque linéairement au début ($\approx$ 108 mJ par round, soit 0,22 % de l'énergie initiale) puis ralentit quand les nœuds meurent ($\approx$ 54 mJ par round entre les rounds 400 et 500) ; elle est quasi nulle dès le round ≈ 800. Le premier nœud meurt au round **191 $\pm$ 14** alors qu'il reste **58 $\pm$ 4 %** de l'énergie du réseau : l'énergie est mal répartie, non épuisée. La période stable (FND/LND) ne représente que 23 % de la durée de vie ; au HND il ne reste que 8 % de l'énergie.



(a) Nombre de CH par round **(b)** Énergie des CH élus et moyenne des vivants (J)

FIGURE 2 – Variabilité du nombre de CH et énergie des CH élus ($N = 100$, 5 graines).

Le nombre de CH par round vaut 4,99 en moyenne (écart-type 2,17 ; de 0 à 13) : 23 rounds sur 948 en ont 10 ou plus et 4 n'en ont aucun, d'où le repli coûteux vers la BS (figure 2a). Les deux courbes de la figure 2b se confondent (rapport énergie des CH / énergie moyenne $= 0{,}96$, mesuré après les dépenses du round donc légèrement sous-estimé) : LEACH n'élit pas de préférence les nœuds riches. Sur la réimplémentation Python (10 graines), la distance à la BS explique l'ordre de mort (corrélation de Spearman $-0{,}94$) : les 10 premiers nœuds morts sont à 243 m de la BS en moyenne, contre 160 m pour l'ensemble des nœuds. Ces deux derniers chiffres ne viennent pas de NS3, dont les CSV ne contiennent pas de données par nœud.

3.2 Expérience B : passage à l'échelle ($N = 50,\ 100,\ 300$)

N	CH/round	Msg/round	Setup (ms)	FND	HND	LND
50	2,6	48,2	26,0	99 ± 16	302 ± 21	797 ± 55
100	5,0	104,3	38,3	191 ± 14	519 ± 29	844 ± 31
300	14,9	314,9	61,0	271 ± 14	638 ± 16	992*

TABLE 2 – Passage à l'échelle. CH, messages et setup : moyennes sur les 30 premiers rounds ; FND/HND/LND en rounds. * LND atteint dans 2 graines sur 5 ; dans les 3 autres, 1 nœud survit au round 1000.

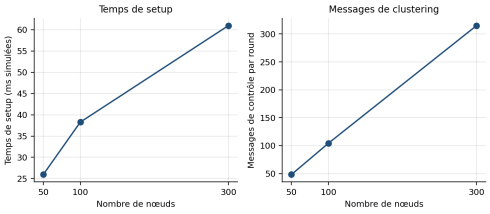


FIGURE 3 – Temps de setup (ms simulées) et messages de clustering par round.

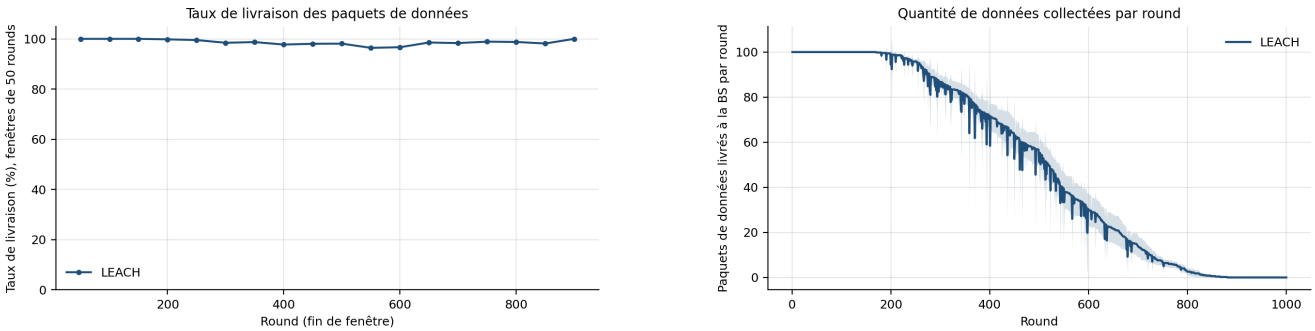
— **Messages de clustering.** Ils valent exactement $N + n_{\text{CH}}$ (ADV : n_{CH} ; JOIN : $N - n_{\text{CH}}$; SCHEDULE : n_{CH}), ce que nous avons vérifié sur 100 % des rounds sans nœud mort et avec $n_{\text{CH}} > 0$: la croissance est linéaire en N. À

$N = 50$ la moyenne (48,2) est inférieure à 52,6 car 8 % des rounds n'ont aucun CH, donc aucun message.

— **Temps de setup.** Il passe de 26 à 38 puis 61 ms ($\times$2,3 pour $\times$6 nœuds, soit $\propto N^{0,48}$). Il vaut ($2n_{\text{CH}}$ + taille du plus gros cluster) $\times$ 0,8 ms (200 bits à 250 kbit/s). Un cluster garde $\approx 1/p = 20$ membres, d'où une croissance sous-linéaire ; mais le terme $2n_{\text{CH}}$ (ADV et SCHEDULE séquentiels) pèse 16 %, 21 % puis 39 % du setup pour $N = 50$, 100, 300 et rend la croissance asymptotiquement linéaire.

— **FND croissant avec** N (99, 191, 271) : à surface fixe, la densité augmente, les clusters sont plus petits et les liaisons membre $\rightarrow$ CH plus courtes (consommation initiale 0,40 ; 0,22 ; 0,18 % de l'énergie initiale par round). Ce n'est pas une amélioration de l'algorithme.

3.3 Expérience C : taux de livraison (paquets de données uniquement)



(a) Taux de livraison (%), fenêtres de 50 rounds **(b)** Paquets livrés à la BS par round

FIGURE 4 – Expérience C : taux de livraison et débit utile ($N = 100$, 5 graines).

Le taux de livraison global est de **98,92 $\pm$ 0,38 %** (de 98,16 à 99,17 % selon la graine) : 50 372 paquets livrés sur 50 916 émis ; les pertes viennent uniquement des CH qui meurent en plein round. Avec 2 % de pertes aléatoires il passe à **95,49 $\pm$ 0,36 %**, valeur cohérente avec deux sauts successifs ($0{,}98^2 \approx 96$ %), et FND/HND/LND restent quasi inchangés (191 / 523 / 840). Le coût moyen est d'environ 1,0 mJ par paquet livré.

Piège d'interprétation. Le taux reste proche de 100 % alors que le réseau s'effondre, car les nœuds morts n'émettent plus et sortent du dénominateur. La figure 4b montre la réalité : 100 paquets livrés par round au début, ≈ 67 vers le round 425, ≈ 10 vers le round 725 et **0** sur les 50 derniers rounds. Il faut donc présenter les deux courbes.

4 Critique de NS3

Facilité d'implémentation. NS3 ne fournit ni LEACH ni clustering natif, et son cadre d'énergie (`EnergySource`, `WifiRadioEnergyModel`), calibré sur des radios réelles, ne correspond pas au modèle imposé : nous avons écrit notre propre comptabilité d'énergie (`TxE/RxE`, une vingtaine de lignes) et la boucle de rounds (`Simulator::Schedule`). Points positifs : `NodeContainer` et `MobilityHelper` (positions), `CommandLine` (paramètres sans recompilation), `RngSeedManager::SetRun` (graines reproductibles), compilation rapide avec trois modules. Points négatifs : courbe d'apprentissage C++ et build (`./ns3 configure/build`, `scratch/`), installation lourde sur Mac ARM (VM + compilation), pas d'outil de tracé intégré.

Réalisme de la couche radio. Notre simulation n'utilise ni la couche physique, ni le MAC, ni la propagation de NS3 : pas de collisions ni d'interférences, TDMA parfait, pas d'énergie d'écoute, et un temps de setup calculé analytiquement (250 kbit/s) et non mesuré ; le taux de livraison ($\approx$ 99 %) est donc optimiste. NS3 propose des modèles réalistes (`lr-wpan`/IEEE 802.15.4, Wi-Fi, propagation log-distance, collisions) pour envoyer de vrais paquets, mais l'énergie ne suivrait plus le modèle imposé sans adaptation et il faudrait compiler davantage de modules ($\approx$ 1 h).

Performances et alternatives. 300 nœuds $\times$ 1000 rounds s'exécutent sans difficulté dans la VM (réglage par défaut du Vagrantfile : 4 Go, 2 vCPU). OMNeT++ (Castalia, INET) apporte des modèles de capteur et de batterie et une interface graphique, au prix d'un langage supplémentaire (NED) ; TOSSIM exécute le vrai code TinyOS mais s'adapte mal à un modèle d'énergie arbitraire (remarques tirées de leur documentation, non utilisés ici).

5 Comparaison avec DEEC

Les résultats du binôme DEEC ne sont pas encore disponibles : la colonne DEEC du tableau 3 est **à compléter** (aucune valeur n'est supposée), avec les mêmes paramètres, graines (1 à 5) et format de CSV, puis `plot_results.py --compare "deec_n100_run*.csv" --label DEEC --other-init 60`.

Métrique ($N = 100$)	**LEACH (NS3)**	**DEEC**	**Attendu**
FND	191 ± 14	*à compléter*	Nettement plus tardif (élection pondérée par $E_{\text{résiduelle}}/E_{\text{moyenne}}$ [3])
HND ; LND	519 ± 29 ; 844 ± 31	*à compléter*	À mesurer
Énergie restante au FND	58 ± 4 %	*à compléter*	CH mieux répartis
Livraison globale	$98{,}92 \pm 0{,}38$ %	*à compléter*	Chute du débit utile plus tardive
Messages / round	104,3	*à compléter*	Même ordre de grandeur

TABLE 3 – Comparaison LEACH / DEEC (5 graines).

Nos résultats motivent déjà cette comparaison : le rapport énergie des CH / énergie moyenne vaut 0,96 (élection sans égard à l'énergie) et la distance à la BS prédit l'ordre de mort. En favorisant les nœuds riches, DEEC devrait retarder le FND. Pour comparer équitablement, il faudra tenir compte des 20 % de nœuds avancés à 1 J de DEEC (60 J contre 50 J) en traçant l'énergie normalisée : une partie de son avantage viendra de ces nœuds et non du seul algorithme.

6 Conclusion

Les simulations confirment les limites de LEACH annoncées par le cours.

1. **Énergie et durée de vie.** Le premier nœud meurt à 23 % de la durée de vie du réseau (FND 191, LND 844) alors que 58 % de l'énergie reste disponible, parce que l'élection ignore énergie et distance ; le nombre de CH fluctue de 0 à 13.
2. **Passage à l'échelle.** Les messages valent exactement $N + n_{\text{CH}}$ (linéaire) et le setup est sous-linéaire ($\propto N^{0,48}$) : LEACH passe bien à l'échelle côté protocole, mais le FND dépend de la densité.
3. **Livraison.** Le taux de ≈ 99 % est trompeur : le débit utile s'effondre de 100 à 0 paquet par round.

LEACH est donc bien « un piège » dans ce scénario où la BS est hors de la zone.

Limites. Canal idéal ; 5 graines seulement ; une seule taille de paquet (la référence Python donne un FND ≈ 206 à 4000 bits, ≈ 406 à 2000 bits et aucun nœud mort en 1000 rounds à 500 bits) ; unité de ε_{mp} à faire valider ; comparaison DEEC à finaliser. **Perspectives :** intégrer les CSV DEEC, passer à 10 graines, et utiliser de vrais paquets (`lr-wpan`) pour obtenir un taux de livraison réaliste.

Références

[1] W. R. Heinzelman, A. Chandrakasan, H. Balakrishnan, « Energy-efficient communication protocol for wireless microsensor networks », *HICSS*, 2000.

[2] W. R. Heinzelman *et al.*, « An application-specific protocol architecture for wireless microsensor networks », *IEEE Trans. Wireless Commun.*, 2002.

[3] L. Qing, Q. Zhu, M. Wang, « Design of a distributed energy-efficient clustering algorithm for heterogeneous wireless sensor networks » (DEEC), *Computer Communications*, 2006.

[4] Documentation NS3, `nsnam.org`.

Annexe

```
== Expérience A : énergie (n=100, 5 graines, 1000 rounds) ==
Graine | FND | HND | LND
  1    | 171 | 508 | 800
  2    | 181 | 569 | 867
  3    | 200 | 479 | 857
  4    | 191 | 518 | 815
  5    | 210 | 520 | 881
LEACH  FND = 191 ± 14
LEACH  HND = 519 ± 29
LEACH  LND = 844 ± 31
Énergie restante au round 1000 : 0.00 J sur 50.0 J
Nombre de CH par round (nœuds tous vivants) : moyenne 4.99, écart-type 2.17, min 0, max 13, rounds sans CH : 4/948

== Expérience C : taux de livraison ==
Taux de livraison global : 98.92 % ± 0.38
Paquets livrés par round : 100 (rounds 1-10) -> 0.0 (50 derniers rounds)
Taux de livraison global avec pertes aléatoires : 95.49 %

== Expérience B : passage à l'échelle ==
N= 50 | setup moyen = 25.952 ms | messages de contrôle/round = 48.2 | CH/round = 2.6 | FND = 99 ± 16 | 5 graines
N=100 | setup moyen = 38.277 ms | messages de contrôle/round = 104.3 | CH/round = 5.0 | FND = 191 ± 14 | 5 graines
N=300 | setup moyen = 60.965 ms | messages de contrôle/round = 314.9 | CH/round = 14.9 | FND = 271 ± 14 | 5 graines
```